

PRACTICAL 1 & 2

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
kvaad@Advaitha:~$ nano practical.c
kvaad@Advaitha:~$ gcc practical.c -o advi
kvaad@Advaitha:~$ ./advi
Enter a Linux command: ls
Parent Process PID: 499
Parent is waiting for Child PID: 500
Child Process PID: 500
Executing command: ls
Hello.c  UserName.c  a.out  advi  practical.c
Child process completed.
Parent Process PID: 499
kvaad@Advaitha:~$ |
```

2. Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

```
kvaad@Advaitha:~$ nano filecopy.c
kvaad@Advaitha:~$ gcc filecopy.c -o advi
kvaad@Advaitha:~$ ./advi
Error opening source file: No such file or directory
kvaad@Advaitha:~$ nano sample.txt
kvaad@Advaitha:~$ cat sample.txt
Operating Systems
Linux provides system calls to applications.
The kernel manages hardware resources.
kvaad@Advaitha:~$ gcc filecopy.c -o filecopy
kvaad@Advaitha:~$ ./filecopy
File copied successfully.
kvaad@Advaitha:~$ cat copy.txt
Operating Systems
Linux provides system calls to applications.
The kernel manages hardware resources.
kvaad@Advaitha:~$ |
```

3. Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution. Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the observations.

```
kvaad@Advaitha:~$ nano process_states.c
kvaad@Advaitha:~$ gcc process_states.c -o process_states
kvaad@Advaitha:~$ ./process_states

--- Before fork() ---
PID : 775
PPID : 323

--- Parent - Running ---
PID : 775
PPID : 323
Parent created child with PID: 776
Parent is waiting for the child to terminate...

--- Child - Running ---
PID : 776
PPID : 775
Child: Going to sleep and enter waiting state...

--- Child - Running Again ---
PID : 776
PPID : 775
Child is terminating...

--- Parent - Child Terminated ---
PID : 775
PPID : 323
Child has terminated.
Parent is now terminating.
kvaad@Advaitha:~$ |
```

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions. Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
kvaad@Advaitha:~$ nano multiple_children.c
kvaad@Advaitha:~$ gcc multiple_children.c -o multiple_children
kvaad@Advaitha:~$ ./multiple_children
Parent PID: 847
Child 1 created: PID = 848, PPID = 847
Child 2 created: PID = 849, PPID = 847
Child 3 created: PID = 850, PPID = 847
Child 1 terminating: PID = 848
Parent: Child PID 848 terminated with exit status 10
Child 2 terminating: PID = 849
Parent: Child PID 849 terminated with exit status 11
Child 3 terminating: PID = 850
Parent: Child PID 850 terminated with exit status 12
Parent: All children have completed.
kvaad@Advaitha:~$ |
```

5. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
kvaad@Advaitha:~$ nano producer_consumer.c
kvaad@Advaitha:~$ gcc producer_consumer.c -o producer_consumer
kvaad@Advaitha:~$ ./producer_consumer
Producer (Parent) PID: 923
Consumer (Child) PID: 924
Producer sent: Data item 1 produced by parent
Consumer received: Data item 1 produced by parent
Producer sent: Data item 2 produced by parent
Consumer received: Data item 2 produced by parent
Producer sent: Data item 3 produced by parent
Consumer received: Data item 3 produced by parent
Producer sent: Data item 4 produced by parent
Consumer received: Data item 4 produced by parent
Producer sent: Data item 5 produced by parent
Consumer received: Data item 5 produced by parent

Consumer finished.

Producer: Consumer has completed.
kvaad@Advaitha:~$ |
```

6. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a POSIX signal handling program that captures `SIGINT`, `SIGTERM`, and `SIGUSR1`. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
kvaad@Advaitha:~$ nano fifo_server.c
kvaad@Advaitha:~$ nano fifo_client.c
kvaad@Advaitha:~$ gcc -Wall -Wextra fifo_server.c -o fifo_server
kvaad@Advaitha:~$ gcc -Wall -Wextra fifo_client.c -o fifo_client
kvaad@Advaitha:~$ ls
fifo_client fifo_client.c fifo_server fifo_server.c
kvaad@Advaitha:~$ ./fifo_server
=====
FIFO SERVER STARTED
=====
Waiting for client...
Client connected.
Client sent: Hello Server
Response sent: Server processed: Hello Server
Client sent: How are you?
Response sent: Server processed: How are you?
Client sent: OS Practical
Response sent: Server processed: OS Practical
|
```

```
kvaad@Advaitha:~$ ./fifo_client
=====
FIFO CLIENT STARTED
=====
Connected to server.

Enter message: Hello Server
Server response: Server processed: Hello Server

Enter message: How are you?
Server response: Server processed: How are you?

Enter message: OS Practical
Server response: Server processed: OS Practical

Enter message: exit
Server response: Server shutting down.
Client terminated.
kvaad@Advaitha:~$ |
```

7. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment. Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
kvaad@Advaitha:~$ nano memory_layout.c
kvaad@Advaitha:~$ gcc -Wall -Wextra memory_layout.c -o memory_layout
kvaad@Advaitha:~$ ./memory_layout
=====
LINUX PROCESS MEMORY LAYOUT
=====

Code address (function):      0x62c748c4b229
Global variable address:     0x62c748c4e010
Global uninitialized address: 0x62c748c4e020
Static global address:       0x62c748c4e014
Static local address:        0x62c748c4e018
Heap variable address:       0x62c759f39010
Stack variable address:      0x7ffed67174bc

=====
Process ID (PID): 492
=====

Process is running...
Use another terminal to inspect /proc/492/maps

Press ENTER to terminate...
|
```

8. Develop a program using `malloc()`, `calloc()`, `realloc()`, and `free()`. Monitor memory allocation behavior and identify memory leaks using Valgrind. Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

```
kvaad@Advaitha:~$ nano dynamic_memory.c
kvaad@Advaitha:~$ gcc -Wall -Wextra dynamic_memory.c -o dynamic_memory
kvaad@Advaitha:~$ ./dynamic_memory
=====
DYNAMIC MEMORY ALLOCATION
=====

1. malloc() demonstration
Values allocated using malloc:
10 20 30 40 50

2. calloc() demonstration
Values immediately after calloc:
0 0 0 0 0
Values after assigning data:
100 200 300 400 500

3. realloc() demonstration
Original arr1 size: 5 integers
After realloc, array contains:
10 20 30 40 50 60 70 80 90 100

4. free() demonstration
arr1 memory released.
arr2 memory released.

All dynamically allocated memory was freed.
kvaad@Advaitha:~$ |
```

9. Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```
kvaad@Advaitha:~$ nano low_level_copy.c
kvaad@Advaitha:~$ gcc low_level_copy.c -o output
kvaad@Advaitha:~$ ./output
Usage: ./output <source> <destination>
kvaad@Advaitha:~$ nano standard_copy.c
kvaad@Advaitha:~$ gcc standard_copy.c
kvaad@Advaitha:~$ gcc standard_copy.c -o output
kvaad@Advaitha:~$ ./output
Usage: ./output <source> <destination>
kvaad@Advaitha:~$ nano redirect_output.c
kvaad@Advaitha:~$ gcc redirect_output.c -o output
kvaad@Advaitha:~$ ./output
kvaad@Advaitha:~$ cat output.txt
This output is redirected to output.txt
stdout is no longer connected to the terminal.
kvaad@Advaitha:~$ nano redirect_input.c
kvaad@Advaitha:~$ gcc redirect_input.c -o output
kvaad@Advaitha:~$ ./output
open: No such file or directory
kvaad@Advaitha:~$ nano mini_shell.c
kvaad@Advaitha:~$ gcc mini_shell.c -o output
kvaad@Advaitha:~$ ./output
Command completed.
Check ls_output.txt
kvaad@Advaitha:~$ cat ls_output.txt
total 292
```

10. Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite). Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```
kvaad@Advaitha:~$ echo "Linux inode demonstration" > original.txt
kvaad@Advaitha:~$ cat original.txt
Linux inode demonstration
kvaad@Advaitha:~$ ls -li original.txt
49946 original.txt
kvaad@Advaitha:~$ ls -li original.txt
49946 -rw-r--r-- 1 kvaad kvaad 26 Sep 28 15:07 original.txt
kvaad@Advaitha:~$ stat original.txt
  File: original.txt
  size: 26          Blocks: 8          IO Block: 4096   regular file
Device: 8,48   Inode: 49946       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   kvaad)   Gid: ( 1000/   kvaad)
Access: 2026-09-28 15:07:20.237248978 +0000
Modify: 2026-09-28 15:07:12.037244432 +0000
Change: 2026-09-28 15:07:12.037244432 +0000
 Birth: 2026-09-28 15:07:12.037244432 +0000
kvaad@Advaitha:~$ find . -inum 123456
kvaad@Advaitha:~$ ./original.txt
-bash: ./original.txt: Permission denied
kvaad@Advaitha:~$ ln original.txt hardlink.txt
kvaad@Advaitha:~$ ls -li
total 304
42076 -rw-r--r-- 1 kvaad kvaad 164 Aug 14 10:06 Multiple_child_process.c
```

```

42034 -rw-r--r-- 1 kvaad kvaad 319 Aug 28 04:06 signal.c
49937 -rw-r--r-- 1 kvaad kvaad 1643 Sep 28 14:53 standard_copy.c
44964 -rw-r--r-- 1 kvaad kvaad 138 Sep 18 03:38 virtual_address.c
kvaad@Advaitha:~$ stat original.txt
  File: original.txt
  Size: 26          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 49946         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/   kvaad)   Gid: ( 1000/   kvaad)
Access: 2026-09-28 15:07:20.237248978 +0000
Modify: 2026-09-28 15:07:12.037244432 +0000
Change: 2026-09-28 15:09:27.861197316 +0000
Birth: 2026-09-28 15:07:12.037244432 +0000
kvaad@Advaitha:~$ stat hardlink.txt
  File: hardlink.txt
  Size: 26          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 49946         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/   kvaad)   Gid: ( 1000/   kvaad)
Access: 2026-09-28 15:07:20.237248978 +0000
Modify: 2026-09-28 15:07:12.037244432 +0000
Change: 2026-09-28 15:09:27.861197316 +0000
Birth: 2026-09-28 15:07:12.037244432 +0000
kvaad@Advaitha:~$ echo "Modified through hard link" >> hardlink.txt
kvaad@Advaitha:~$ cat original.txt
Linux inode demonstration
Modified through hard link
kvaad@Advaitha:~$ rm original.txt
kvaad@Advaitha:~$ ls -li
total 300
42076 -rw-r--r-- 1 kvaad kvaad 164 Aug 14 10:06 Multiple_child_process.c
42088 -rw-r--r-- 1 kvaad kvaad 152 Aug 14 10:14 Parent_child_executing.c
42032 -rw-r--r-- 1 kvaad kvaad 258 Aug 14 09:59 Parent_waits_for_child.c
49928 -rwxr-xr-x 1 kvaad kvaad 16400 Sep 28 14:53 a.out
51656 -rw-r--r-- 1 kvaad kvaad 499 Sep 18 04:07 address_translation.c

```

```

49940 -rw-r--r-- 1 kvaad kvaad 541 Sep 28 14:54 redirect_output.c
51689 -rw-r--r-- 1 kvaad kvaad 102 Sep 28 14:18 sample.txt
42034 -rw-r--r-- 1 kvaad kvaad 319 Aug 28 04:06 signal.c
49937 -rw-r--r-- 1 kvaad kvaad 1643 Sep 28 14:53 standard_copy.c
44964 -rw-r--r-- 1 kvaad kvaad 138 Sep 18 03:38 virtual_address.c
kvaad@Advaitha:~$ echo "Linux symbolic link demonstration" > original.txt
kvaad@Advaitha:~$ ln -s original.txt symlink.txt
kvaad@Advaitha:~$ ls -li
total 304
42076 -rw-r--r-- 1 kvaad kvaad 164 Aug 14 10:06 Multiple_child_process.c
42088 -rw-r--r-- 1 kvaad kvaad 152 Aug 14 10:14 Parent_child_executing.c
42032 -rw-r--r-- 1 kvaad kvaad 258 Aug 14 09:59 Parent_waits_for_child.c
49928 -rwxr-xr-x 1 kvaad kvaad 16400 Sep 28 14:53 a.out
51656 -rw-r--r-- 1 kvaad kvaad 499 Sep 18 04:07 address_translation.c
42012 -rwxr-xr-x 1 kvaad kvaad 16168 Aug 28 04:06 advi
51628 -rw-r--r-- 1 kvaad kvaad 921 Sep 28 14:05 command_executor.c
51690 -rw-r--r-- 1 kvaad kvaad 102 Sep 28 14:19 copy.txt
51655 -rw-r--r-- 1 kvaad kvaad 350 Sep 18 03:43 demand_page.c
49938 -rw-r--r-- 1 kvaad kvaad 2506 Sep 28 14:45 dynamic_layout.c
42017 -rwxr-xr-x 1 kvaad kvaad 16216 Aug 12 17:14 dynamic_memory
42011 -rw-r--r-- 1 kvaad kvaad 2506 Aug 12 16:51 dynamic_memory.c
49936 -rw----- 1 kvaad kvaad 4 Sep 28 14:36 fifo.save
41995 -rwxr-xr-x 1 kvaad kvaad 16648 Aug 12 03:32 fifo_client
41975 -rw-r--r-- 1 kvaad kvaad 2546 Aug 12 03:31 fifo_client.c

```

```

44964 -rw-r--r-- 1 kvaad kvaad 138 Sep 18 03:38 Virtual_address.c
kvaad@Advaitha:~$ stat symlink.txt
  File: 'symlink.txt' -> 'original.txt'
  size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48      Inode: 50755         Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/   kvaad)   Gid: ( 1000/   kvaad)
Access: 2026-09-28 15:13:32.485164155 +0000
Modify: 2026-09-28 15:13:28.473164667 +0000
Change: 2026-09-28 15:13:28.473164667 +0000
 Birth: 2026-09-28 15:13:28.473164667 +0000
kvaad@Advaitha:~$ stat -L symlink.txt
  File: symlink.txt
  size: 34          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 49950         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   kvaad)   Gid: ( 1000/   kvaad)
Access: 2026-09-28 15:13:14.629166082 +0000
Modify: 2026-09-28 15:13:14.629166082 +0000
Change: 2026-09-28 15:13:14.629166082 +0000
 Birth: 2026-09-28 15:13:14.629166082 +0000
kvaad@Advaitha:~$ find . -inum $(ls -li symlink.txt | awk '{print $1}')
./symlink.txt
kvaad@Advaitha:~$ find . -type l -ls
 50755      0 lrwxrwxrwx  1 kvaad    kvaad          12 Sep 28 15:13 ./symlink.txt
kvaad@Advaitha:~$ nano nmap_file.c
kvaad@Advaitha:~$ gcc -Wall -Wextra nmap_file.c -o nmap_file
kvaad@Advaitha:~$ ./nmap_file
Original file content:
Hello from memory mapped I/O!

Modified content:
Modified using mmap()!
d I/O!

Changes written to mmap_test.txt
kvaad@Advaitha:~$ cat nmap_test.txt
cat: nmap_test.txt: No such file or directory
kvaad@Advaitha:~$ nano traditional_io.c
kvaad@Advaitha:~$ gcc -Wall -Wextra traditional_io.c -o traditional_io
kvaad@Advaitha:~$ ./traditional_io
Original content:

```

```

kvaad@Advaitha:~$ cat nmap_test.txt
cat: nmap_test.txt: No such file or directory
kvaad@Advaitha:~$ nano traditional_io.c
kvaad@Advaitha:~$ gcc -Wall -Wextra traditional_io.c -o traditional_io
kvaad@Advaitha:~$ ./traditional_io
Original content:
Hello from traditional read/write I/O!

Modified content:
Modified using read/write!
kvaad@Advaitha:~$ |

```

11. Develop a multithreaded counter application using POSIX threads (`pthread_create`, `pthread_join`). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization

```

kvaad@Advaitha:~$ time ./race_condition
Starting race-condition test...

Expected counter value : 4000000
Actual counter value   : 1950983

real    0m0.016s
user    0m0.036s
sys     0m0.000s
kvaad@Advaitha:~$ time ./mutex_counter
Starting mutex-protected test...

Expected counter value : 4000000
Actual counter value   : 4000000

real    0m0.272s
user    0m0.390s
sys     0m0.394s

```

12. Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes. Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```

kvaad@Advaitha:~$ nano producer_consumer.c
kvaad@Advaitha:~$ gcc -Wall -Wextra -pthread producer_consumer.c
-o producer_consumer
producer_consumer.c: In function 'producer':
producer_consumer.c:25:22: warning: unused parameter 'arg' [-Wunused-parameter]
    25 | void *producer(void *arg)
        |                   ^~~~~~
producer_consumer.c: In function 'consumer':
producer_consumer.c:60:22: warning: unused parameter 'arg' [-Wunused-parameter]
    60 | void *consumer(void *arg)
        |                   ^~~~~~
kvaad@Advaitha:~$ ./producer_consumer 5

=====
  Producer-Consumer Results
=====
Buffer size      : 5
Producers        : 2
Consumers        : 2
Expected items   : 200000
Produced items   : 200000
Consumed items   : 200000
Execution time    : 1.984862 seconds
Throughput       : 100762.69 items/sec
Synchronization  : CORRECT
kvaad@Advaitha:~$

```

```

kvaad@Advaitha:~$ nano deadlock.c
kvaad@Advaitha:~$ gcc -Wall -Wextra -pthread deadlock.c -o deadlock
deadlock.c: In function 'thread1':
deadlock.c:11:21: warning: unused parameter 'arg' [-Wunused-parameter]
    11 | void *thread1(void *arg)
        |                  ^~~~~
deadlock.c: In function 'thread2':
deadlock.c:35:21: warning: unused parameter 'arg' [-Wunused-parameter]
    35 | void *thread2(void *arg)
        |                  ^~~~~
kvaad@Advaitha:~$ ./deadlock
Starting deadlock demonstration...

Thread 2: trying to lock Resource B
Thread 2: locked Resource B
Thread 1: trying to lock Resource A
Thread 1: locked Resource A
Thread 2: trying to lock Resource A
Thread 1: trying to lock Resource B
^C
kvaad@Advaitha:~$ |

```

```

kvaad@Advaitha:~$ ./producer_consumer 1
=====
Producer-Consumer Results
=====
Buffer size      : 1
Producers        : 2
Consumers        : 2
Expected items   : 200000
Produced items   : 200000
Consumed items   : 200000
Execution time    : 11.592603 seconds
Throughput       : 17252.38 items/sec
Synchronization : CORRECT
kvaad@Advaitha:~$ ./producer_consumer 10
=====
Producer-Consumer Results
=====
Buffer size      : 10
Producers        : 2
Consumers        : 2
Expected items   : 200000
Produced items   : 200000
Consumed items   : 200000
Execution time    : 0.913388 seconds
Throughput       : 218965.03 items/sec
Synchronization : CORRECT

```

```

kvaad@Advaitha:~$ nano deadlock_prevented.c
kvaad@Advaitha:~$ gcc -Wall -Wextra -pthread deadlock_prevented.
c -o deadlock_prevented
deadlock_prevented.c: In function 'thread1':
deadlock_prevented.c:11:21: warning: unused parameter 'arg' [-Wu
nused-parameter]
   11 | void *thread1(void *arg)
       |                  ~~~~~^~
deadlock_prevented.c: In function 'thread2':
deadlock_prevented.c:37:21: warning: unused parameter 'arg' [-Wu
nused-parameter]
   37 | void *thread2(void *arg)
       |                  ~~~~~^~
kvaad@Advaitha:~$ ./deadlock_prevented
Starting deadlock-prevention demonstration...

Thread 2: trying to lock Resource A
Thread 2: locked Resource A
Thread 1: trying to lock Resource A
Thread 2: trying to lock Resource B
Thread 2: locked Resource B
Thread 2: using both resources
Thread 1: locked Resource A
Thread 2: released both resources
Thread 1: trying to lock Resource B
Thread 1: locked Resource B
Thread 1: using both resources
Thread 1: released both resources

Program completed successfully.
kvaad@Advaitha:~$ |

```